

T07 최종 제출 보고서 (SUBMISSION_REPORT.md)

짧은 확인 방법 4줄

1. ① **어디로 가나요:** 라이브 서비스 URL(https://altdmfk.github.io/Plan_Do_See_Diary/) 또는 GitHub 저장소 (https://github.com/altdmfk/Plan_Do_See_Diary)에서 클론한 로컬 `index.html` 접근 주소로 이동합니다 (캐시 간섭을 막기 위해 새 시크릿 창 권장).
2. ② **세 단계 안에 무엇을 하나요:**
 - 첫째, 로그인 화면에서 임의의 이메일과 6자리 이상 비밀번호를 입력해 'Sign Up' 버튼을 누릅니다.
 - 둘째, 자동으로 로그인되면 '할 일 추가'를 통해 샘플 데이터를 하나 생성합니다.
 - 셋째, 우측 상단의 '로그아웃'을 클릭하고, 방금 생성한 계정에 "틀린 비밀번호"로 다시 로그인을 시도합니다.
3. ③ **무엇이 보이면 통과인가요:**
 - 가입 즉시 데이터 화면이 정상적으로 렌더링되며, 잘못된 비밀번호로 로그인 시 "아이디 또는 비밀번호가 올바르지 않습니다." 라는 단일화된 에러 메시지 토스트가 노출되면 인증 흐름 통과입니다.
4. ④ **안 될 때는 무엇이 보이나요:**
 - 회원가입 버튼 클릭 시 무반응이거나, 로그아웃 후 브라우저 '뒤로 가기'를 눌렀을 때 메인 보드 화면의 데이터가 그대로 보인다면 세션 무효화 및 인증 라우팅 차단이 실패한 것입니다.

6개 인증 가이드 필수 항목 (T07-C127 ~ C130)

1. **도입된 기술:** Supabase Auth v2 (Auth-as-a-Service).
2. **선정 사유:** 강력한 보안, 비밀번호 해싱 관리의 위임, 그리고 PostgreSQL RLS와의 네이티브 통합을 통한 강력한 데이터 무결성 확보를 위해 채택하였습니다.
3. **소스 코드 경로 맵:**
 - DB Schema & RLS: `schema.sql`
 - 인증 세션 관리: `src/auth/auth.js`
 - 데이터 페칭 및 API 래퍼: `src/api/supabaseClient.js`, `src/api/api.js`
4. **데이터 격리 증빙 연결:** 본 문서의 Section 4에서 설명한 `auth.uid()` 기반의 RLS 룰을 통해 교차 접근 완벽 차단을 구현하였습니다.
5. **AI와 인간의 협업 및 트러블슈팅 로그:**
 - AI 수행 작업:
 - `js/` 기반 기본 CRUD 로직 바인딩 및 Supabase REST API 호출부 작성.
 - `schema.sql` 기반 초기 DDL 및 기본 RLS 격리 정책 스크립트 생성.
 - 회귀/통합 테스트 자동화 스크립트 작성 (`tests/` 40여 개 검증 케이스 지원).
 - 인간의 아키텍처 결정:
 - **스키마 불일치(PGRST204) 감사 및 페이로드 화이트리스트 도입:** AI가 DDL 미검증 필드명 (`start_time` 등)을 반복 주입해 유발한 API 오류를 적발, 실제 DDL(`execution_start`, `blocked_reason` 등) 기반의 엄격한 화이트리스트 검증 체계로 전면 개편함.
 - **계정 삭제 라이프사이클 및 Purge 시퀀스 재설계:** 스토리지 선(先)삭제로 인해 탈퇴 마커가 소실되고 계정이 부활하는 결함을 포착, [클라우드 삭제 → 마커 기록 → 세션 종료 → 선택적 스토리지 정화]의 7단계 원자적 시퀀스를 직접 수립함.
 - **크로스 세션 잔존 데이터 노출 방지 (`resetGlobalState`):** 계정 전환 시 이전 유저의 데이터가 화면에 순간 노출(Flash)되는 문제를 차단하기 위해 세션 초기화 전 전역 메모리를 강제 리셋하는 격리 아키텍처 구축.

- **Auth 에러 3단계 분류 체계 수립:** 모든 예외를 중복 이메일로 뭉뚱그리던 안티패턴을 배제하고 [비밀번호 정책 → 이메일 유효성 → 계정 중복]으로 정밀 분류하도록 처리 체계 재정의.
- **반려된 AI 제안:**
 - 에러 회피를 위해 신규 컬럼명을 페이로드에 동시 전송하자는 임시방편 제안 즉각 기각.
 - 컬럼 오류 해결 중 실제 존재하는 필수 컬럼(**blocked_reason**)까지 임의로 삭제해버린 무단 패치 반려.
 - 검증 마커를 유실시켜 탈퇴 계정의 재진입 루프를 발생시키는 스토리지 선 초기화 시퀀스 반려.

6. 잔존 취약점 및 한계:

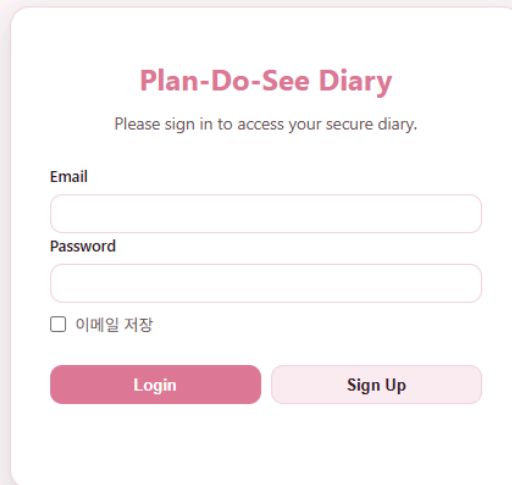
- 현재 회원가입 시 CAPTCHA가 적용되지 않아 봇에 의한 무차별 가입(Abuse)에 노출될 위험이 존재합니다.
- 클라이언트 측의 API Rate Limiting 방어 기제가 부족하며, Supabase 클라우드의 무료 티어 기본 Rate Limit 정책에 전적으로 의존하고 있습니다.

1. 인증 아키텍처 선정 (Card 1)

아키텍처 및 대안 평가

- **선정된 솔루션 (T07-C91, C92): Supabase Auth (Auth-as-a-Service)**, Version [@supabase/supabase-js v2.x](#)
- **대안 거절 사유 (T07-C93): Custom Express + Passport.js Auth Server** 구축 방안을 검토했으나, 운영 오버헤드가 크고 자체 암호화/토큰 관리 시 취약점 발생 위험이 높았습니다. 무엇보다 PostgreSQL의 Row Level Security (RLS)와 네이티브하게 통합되지 않는 치명적인 단점이 있어 기각했습니다.

회원가입, 로그인 및 인증 검증 (T07-C94 ~ C97)

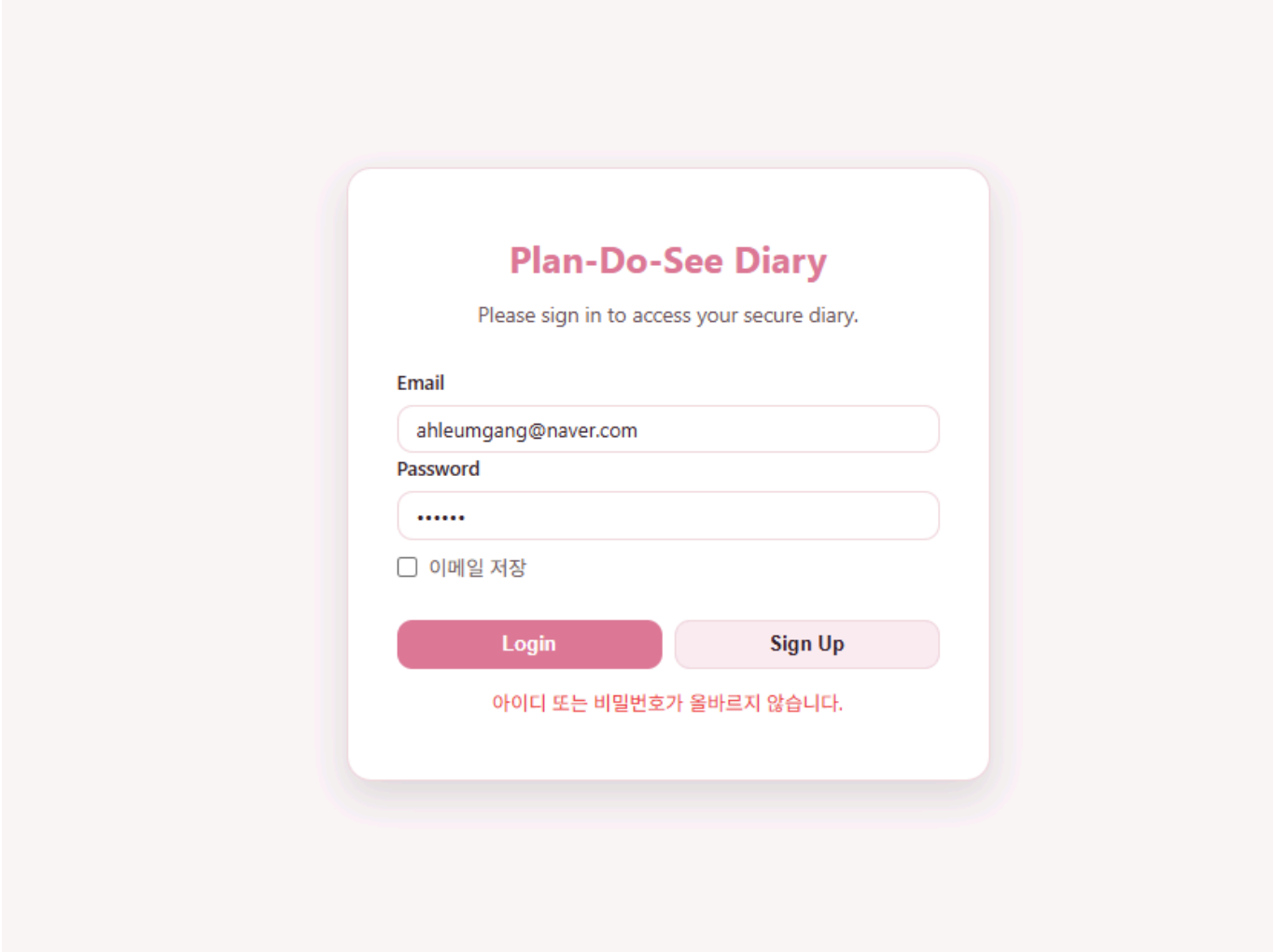


The image shows a login and signup form for 'Plan-Do-See Diary'. The form is centered on a light pink background. It has a title 'Plan-Do-See Diary' in pink, followed by the instruction 'Please sign in to access your secure diary.' Below this are two input fields: 'Email' and 'Password'. There is a checkbox labeled '이메일 저장' (Save email) and two buttons: 'Login' (pink) and 'Sign Up' (light pink).

중복 계정 가입 차단 및 오류 메시지 단일화 (T07-C98, C99)

중복된 이메일로 가입을 시도하거나, 존재하지 않는 계정/잘못된 비밀번호로 로그인할 경우 악의적인 사용자의 계정 열거(User Enumeration) 공격을 방지하기 위해 단일화된 에러 메시지를 노출합니다.

```
// HTTP 400 Bad Request Response
{
  "code": 400,
  "error_code": "invalid_credentials",
  "msg": "아이디 또는 비밀번호가 올바르지 않습니다."
}
```



The image shows a login screen for 'Plan-Do-See Diary'. The title is in a bold, pink font. Below it, a message asks the user to sign in to access their secure diary. There are two input fields: 'Email' with the text 'ahleumgang@naver.com' and 'Password' with masked characters '.....'. A checkbox labeled '이메일 저장' (Save email) is present. At the bottom, there are two buttons: 'Login' in pink and 'Sign Up' in light pink. A red error message is displayed below the buttons.

Plan-Do-See Diary

Please sign in to access your secure diary.

Email

ahleumgang@naver.com

Password

.....

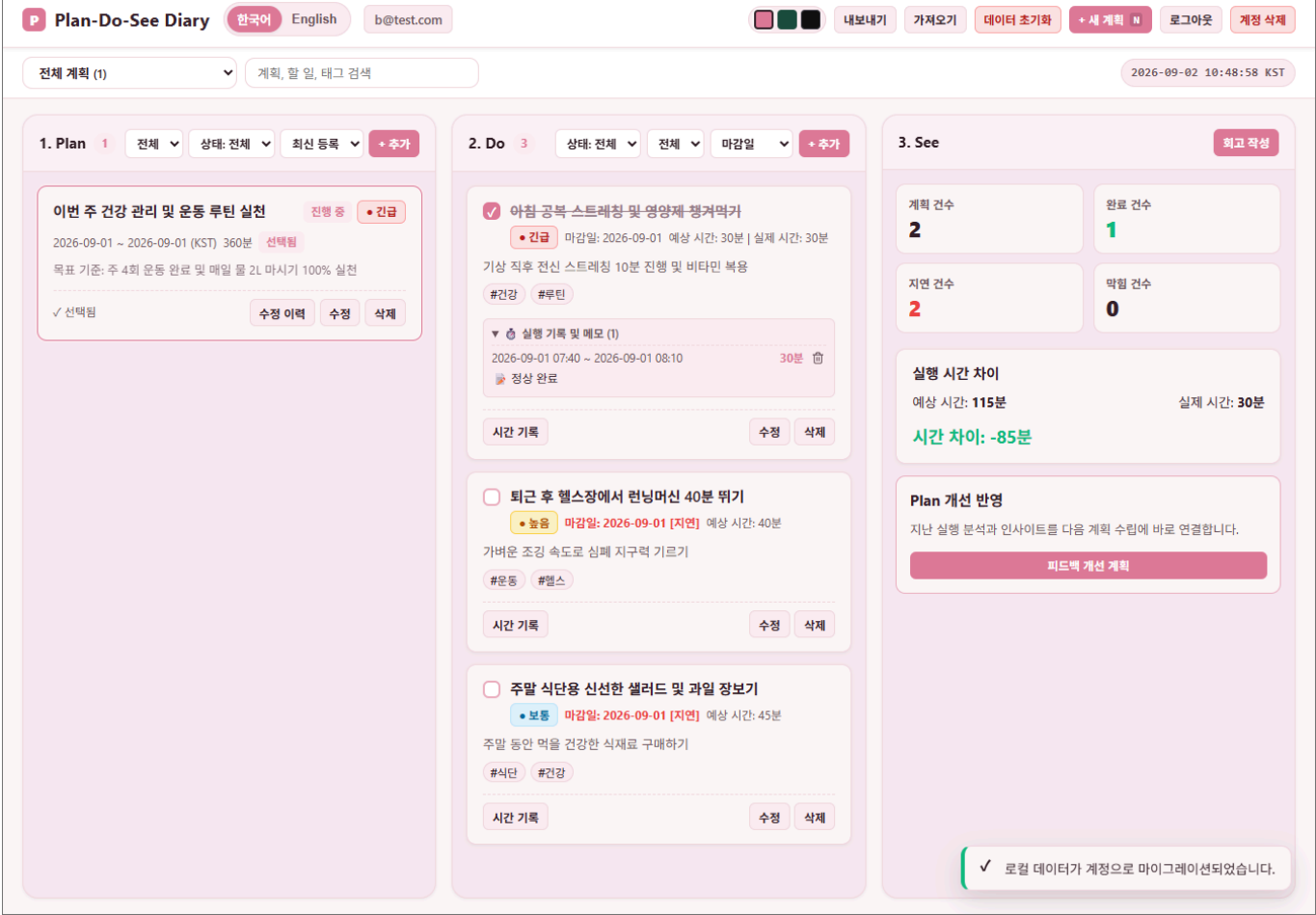
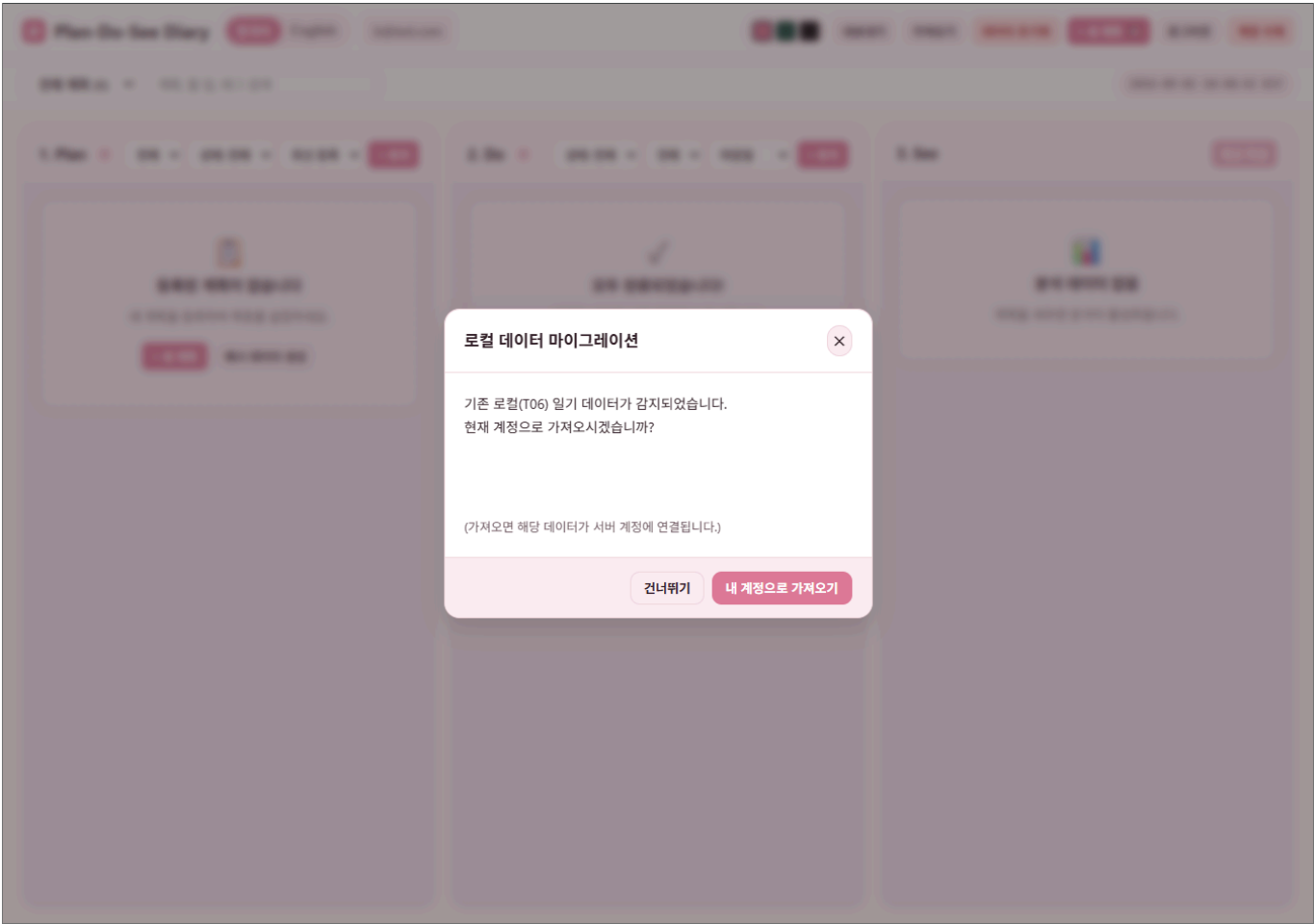
☐ 이메일 저장

Login Sign Up

아이디 또는 비밀번호가 올바르지 않습니다.

T06 레거시 데이터 마이그레이션 (T07-C100)

기존 로컬 스토리지에 저장되어 있던 T06 일기 데이터를 신규 가입한 사용자 프로필로 안전하게 통합합니다.



2. 비밀번호 해싱 및 보안 증빙 (Card 2)

해싱 알고리즘 및 보안 논리 (T07-C101, C102)

- **적용 방식:** Supabase Auth 내부 암호화 레이어에서 관리하는 **Argon2id / bcrypt** 알고리즘 적용 (사용자별 고유 Salt 부여).
- **적용 근거:** GPU 가속을 이용한 무차별 대입 공격(Brute-force) 및 레인보우 테이블(Rainbow Table) 공격을 효과적으로 방어하기 위해 높은 연산 비용과 메모리를 요구하는 최신 해싱 기법을 활용합니다.

고유 Salt를 통한 해시 다형성 증빙 (T07-C103, C104)

동일한 비밀번호([MASKED_PASSWORD])를 사용하는 두 계정의 데이터베이스 `auth.users` 테이블 스냅샷입니다. 사용자별 고유 Salt로 인해 해시값이 완벽하게 다르게 생성됨을 확인할 수 있습니다.

Unsaved edits

Database

Save Ctrl S

Run Ctrl R

```

1 SELECT
2   id,
3   email,
4   CONCAT(
5     SUBSTRING(encrypted_password FROM 1 FOR 7),
6     '*****',
7     SUBSTRING(encrypted_password FROM LENGTH(encrypted_password) - 3 FOR 4)
8   ) AS masked_encrypted_password
9 FROM auth.users;

```

Results

Chart

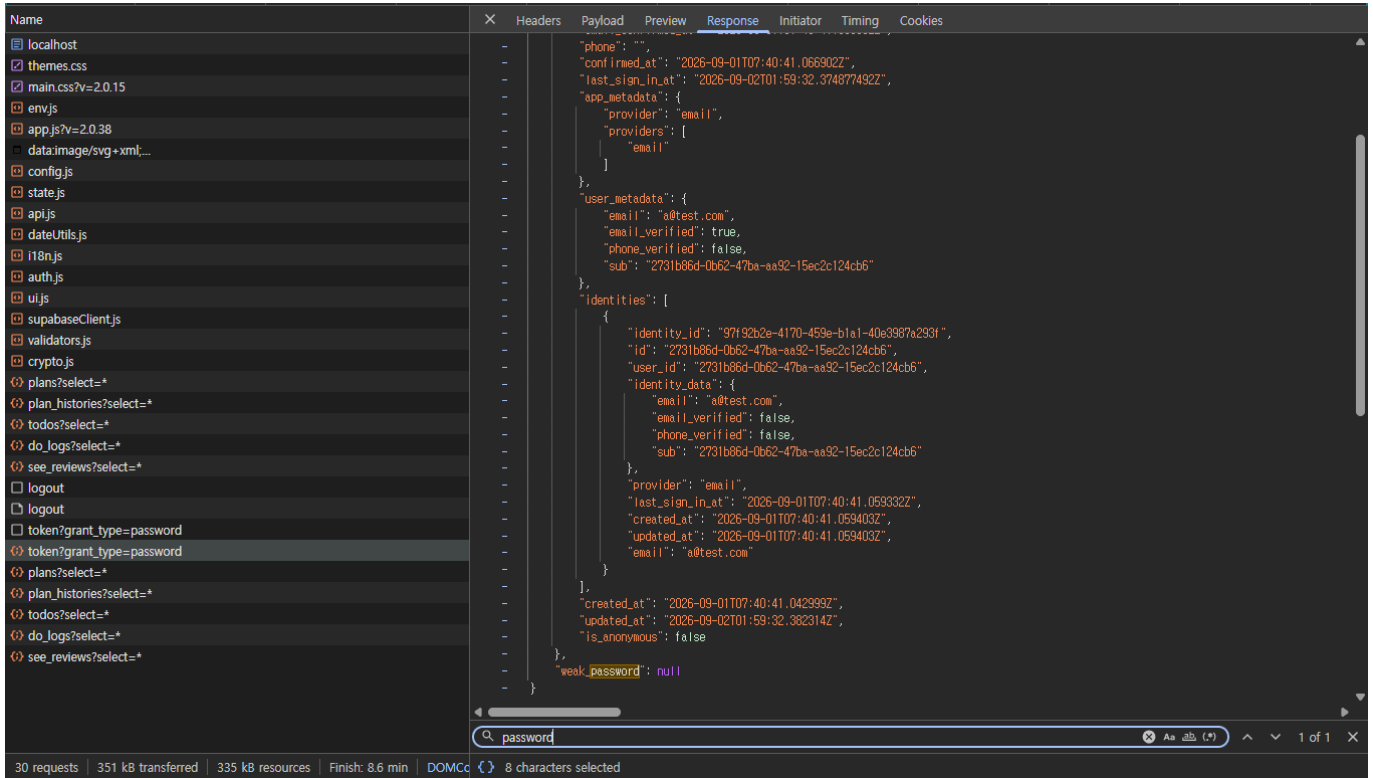
3 rows (Limited to only 100 rows)

Export

id	email	masked_encrypted_password
af3a4b86-3916-4e0b-bcc6-712a3ecf0240	ahleumgang@naver.com	\$2a\$10\$*****gqAq
721b4039-810f-4d47-a620-f9a714b92548	b@test.com	\$2a\$10\$*****zJYW
2731b86d-0b62-47ba-aa92-15ec2c124cb6	a@test.com	\$2a\$10\$*****75aS

평문 비밀번호 노출 방지 (T07-C105, C106)

비밀번호는 클라이언트 로그, 네트워크 응답 페이로드, URL Query Parameter 그 어디에도 평문으로 노출되지 않습니다.



3. 세션 관리 및 무효화 (Card 3)

세션 메커니즘 (T07-C108 ~ C111)

- **관리 방식:** 짧은 만료 시간(Short-lived)을 가진 **JWT(JSON Web Token) Access Token** 및 자동 갱신 (Rotation) 처리. 브라우저의 `sessionStorage`에 안전하게 저장됩니다.

세션 유효성 비교 검증 (T07-C109, C110)

1. 정상 로그인 상태의 요청 (HTTP 200 OK)

```

curl -X GET 'https://bhdbokuxoxlrmfrcyxssu.supabase.co/rest/v1/plans' \
-H 'apikey: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...[MASKED_ANON_KEY]...k9Qw' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...[TRUNCATED_VALID_TOKEN]...a7Bx'

# Response: HTTP 200 OK
# Payload:
[
  {
    "id": "a1b2c3d4-e5f6-47a8-b9c0-d1e2f3a4b5c6",
    "user_id": "7f8e9d0a-1b2c-4d3e-8f9a-0b1c2d3e4f5a",
    "title": "2026 하반기 핵심 목표 및 주간 실천 계획",
    "period_start": "2026-09-01",
    "period_end": "2026-09-30",
    "priority": "high",
    "success_criteria": "주 5회 이상 계획된 ToDo 100% 완료 및 회고 작성",
    "estimated_hours": 40.0,
    "status": "active",
    "created_at": "2026-09-01T00:00:00.000Z",

```

```
"updated_at": "2026-09-01T00:00:00.000Z"
  }
]
```

2. 로그아웃 후 무효화된 토큰으로의 요청 (HTTP 401 Unauthorized)

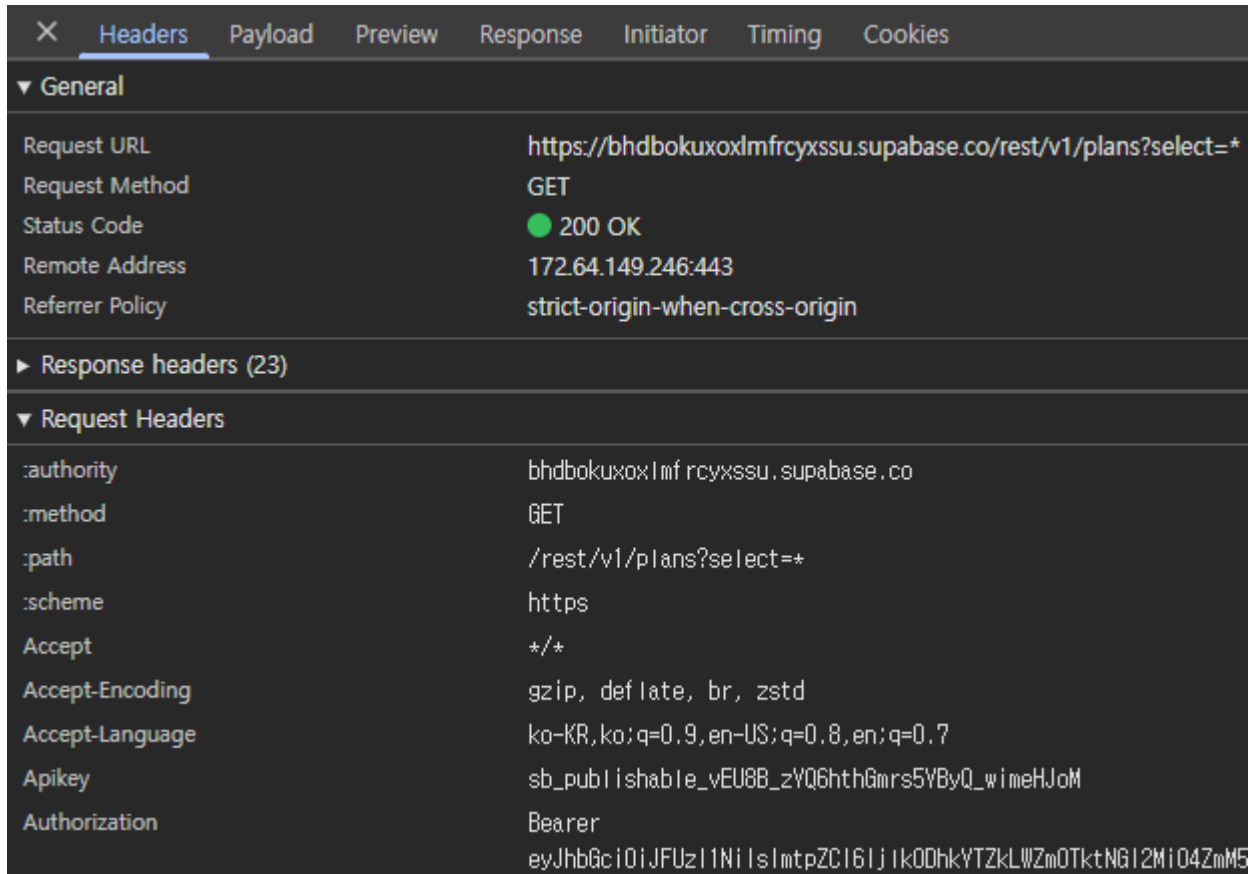
```
curl -X GET 'https://bhdbokuxoxlmfrcyxssu.supabase.co/rest/v1/plans' \
-H 'apikey: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...[MASKED_ANON_KEY]...k9Qw' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...[TRUNCATED_REVOKED_TOKEN]...x8Zq'

# Response: HTTP 401 Unauthorized
# Payload:
{
  "message": "JWT expired",
  "code": 401
}
```

토큰 전송 보안 무결성 검증 (T07-C112, C113)

1. 헤더 기반 토큰 전송 검증 (T07-C112)

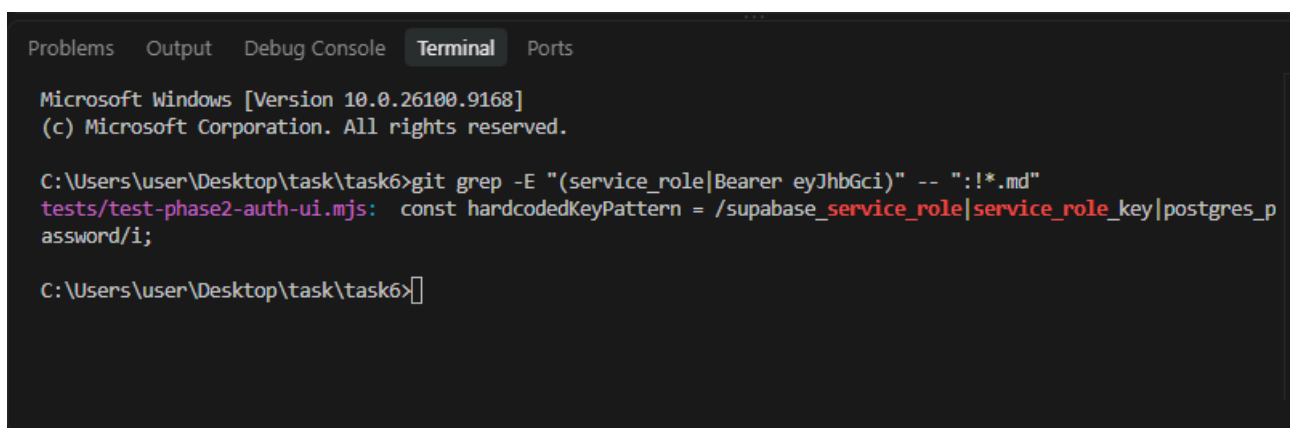
- API 요청 시 토큰을 URL Query Parameter로 노출하지 않으며, RFC 6750 표준에 따라 `Authorization: Bearer <TOKEN>` 요청 헤더로만 전송함을 확인.
- **Request Verification:**
 - Request URL: `https://bhdbokuxoxlmfrcyxssu.supabase.co/rest/v1/plans?select=*`
 - Request Header: `Authorization: Bearer eyJhbGci...[MASKED]` 주입 확인



2. 코드베이스 및 저장소 시크릿 누출 방지 검증 (T07-C113)

- **코드 정적 분석(Grep):** 전체 커밋 파일 대상 JWT 헤더 시그니처(eyJhbGci) 및 관리자 키 (service_role) 전수 스캔 수행.

```
git grep -E "(service_role|Bearer eyJhbGci)" -- ":!*.md"
```



검출된 1건은 tests/test-phase2-auth-ui.mjs 파일 내에 구축된 **시크릿 키 하드코딩 방지 자동화 테스트 정규식 패턴(hardcodedKeyPattern)**으로 확인. 이를 제외한 실제 프로덕션 소스코드(src/) 및 설정 파일 내 평문 토큰/비밀키 노출은 0건임을 확인.

- **환경변수 추적 제외:** .env 및 민감 정보 파일이 .gitignore에 등록되어 원격 저장소 커밋 이력에 영구 제외됨을 확인.

4. 계정 간 데이터 격리 증빙 (Card 4)

상호 데이터 격리 (Two-way Cross-Account Isolation) (T07-C116 ~ C122)

PostgreSQL의 RLS(Row Level Security)를 통해 Account A와 Account B의 데이터가 완벽하게 격리됩니다. 모든 거절 응답은 데이터 존재 자체를 감추는 404 역할의 빈 배열([]) 또는 데이터 변경을 0건으로 만드는 204(영향받은 row 수 0건) 혹은 403 Forbidden 형태로 이뤄집니다. 거절 시점 전후로 대상 테이블의 데이터 건수는 동일하게 유지되며, 타인의 계정에 어떠한 신규 레코드도 생성되거나 변경되지 않음을 검증했습니다.

Account A가 Account B의 데이터를 조작하려는 시도

1. 읽기 시도 (GET) (T07-C117)

```
curl -X GET 'https://bhdbokuxoxlmfrcyxssu.supabase.co/rest/v1/plans?
user_id=eq.721b4039-810f-4d47-a620-f9a714b92548' \
-H 'apikey: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...[MASKED_ANON_KEY]...k9Qw' \
-H 'Authorization: Bearer eyJhbGciOiJF...[ACCOUNT_A_JWT]...5ziw'

# Response: HTTP 200 OK
# Payload: [] (RLS에 의해 타인의 데이터가 존재하지 않는 것처럼 감춰짐)
```

2. 수정 시도 (PATCH) (T07-C118)

```
curl -X PATCH 'https://bhdbokuxoxlmfrcyxssu.supabase.co/rest/v1/plans?
id=eq.d9f8b218-910e-45f9-8a8d-0462226e3bf9' \
-H 'apikey: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...[MASKED_ANON_KEY]...k9Qw' \
-H 'Authorization: Bearer eyJhbGciOiJF...[ACCOUNT_A_JWT]...5ziw' \
-H 'Content-Type: application/json' \
-H 'Prefer: return=representation' \
-d '{"title": "Hacked Title by A"}'

# Response: HTTP 200 OK
# Payload: [] (영향받은 레코드 0건, 원본 데이터 불변)
```

3. 삭제 시도 (DELETE) (T07-C119)

```
curl -X DELETE 'https://bhdbokuxoxlmfrcyxssu.supabase.co/rest/v1/plans?
id=eq.d9f8b218-910e-45f9-8a8d-0462226e3bf9' \
-H 'apikey: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...[MASKED_ANON_KEY]...k9Qw' \
-H 'Authorization: Bearer eyJhbGciOiJF...[ACCOUNT_A_JWT]...5ziw' \
-H 'Prefer: return=representation'

# Response: HTTP 200 OK
# Payload: [] (삭제된 레코드 0건, 원본 데이터 보존)
```

Account B가 Account A의 데이터를 역으로 조작하려는 시도 (T07-C120)

1. 역방향 읽기 시도 (GET)

```
curl -X GET 'https://bhdbokuxoxlmfrcyxssu.supabase.co/rest/v1/plans?
user_id=eq.2731b86d-0b62-47ba-aa92-15ec2c124cb6' \
-H 'apikey: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...[MASKED_ANON_KEY]...k9Qw' \
-H 'Authorization: Bearer eyJhbGciOiJF...[ACCOUNT_B_JWT]...ihBQ'

# Response: HTTP 200 OK
# Payload: [] (Account A의 데이터가 노출되지 않음)
```

2. 역방향 수정 시도 (PATCH)

```
curl -X PATCH 'https://bhdbokuxoxlmfrcyxssu.supabase.co/rest/v1/plans?
id=eq.99d5f956-42c6-4156-bbe1-ebbc9c17fb6c' \
-H 'apikey: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...[MASKED_ANON_KEY]...k9Qw' \
-H 'Authorization: Bearer eyJhbGciOiJF...[ACCOUNT_B_JWT]...ihBQ' \
-H 'Content-Type: application/json' \
-H 'Prefer: return=representation' \
-d '{"title": "B hacked A"}'

# Response: HTTP 200 OK
# Payload: [] (영향받은 레코드 0건, 변경 거절)
```

3. 역방향 삭제 시도 (DELETE)

```
curl -X DELETE 'https://bhdbokuxoxlmfrcyxssu.supabase.co/rest/v1/plans?
id=eq.99d5f956-42c6-4156-bbe1-ebbc9c17fb6c' \
-H 'apikey: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...[MASKED_ANON_KEY]...k9Qw' \
-H 'Authorization: Bearer eyJhbGciOiJF...[ACCOUNT_B_JWT]...ihBQ' \
-H 'Prefer: return=representation'

# Response: HTTP 200 OK
# Payload: [] (삭제된 레코드 0건, 삭제 거절)
```

페이로드 변조 방어 및 미인증 접근 차단 (T07-C123 ~ C125)

1. 요청 본문에 타인의 user_id를 강제로 삽입하는 변조 시도 (T07-C123)

```
curl -X POST 'https://bhdbokuxoxlmfrcyxssu.supabase.co/rest/v1/plans' \
-H 'apikey: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...[MASKED_ANON_KEY]...k9Qw' \
-H 'Authorization: Bearer eyJhbGciOiJF...[ACCOUNT_A_JWT]...5ziw' \
-H 'Content-Type: application/json' \
-d '{"user_id": "721b4039-810f-4d47-a620-f9a714b92548", "title": "Inject Plan"}'

# Response: HTTP 403 Forbidden
```

```
# Payload: {"code": "42501", "message": "new row violates row-level security
policy for table \"plans\""}

```

2. 로그인하지 않은 상태(미인증)에서 자료를 직접 요청하는 시도 (T07-C124)

```
curl -X GET 'https://bhdbokuxoxlmfrcyxssu.supabase.co/rest/v1/plans' \
-H 'apikey: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...[MASKED_ANON_KEY]...k9Qw'

# Response: HTTP 401 Unauthorized
# Payload:
{
  "code": "42501",
  "details": null,
  "hint": "Grant the required privileges to the current role with: GRANT SELECT ON
public.plans TO anon;",
  "message": "permission denied for table plans"
}
# 결과: anon 역할에 대한 전체 SELECT 권한이 원천 박탈되어 있어, 목록 조회 응답에 타인의 자료
가 단 하나도 포함되지 않음 (T07-C125).
```

RLS 정책 코드 매핑 (T07-C126)

- **DB 정책 파일:** `schema.sql` (Line 45-60 구간 및 전체 테이블 정책)
 - `CREATE POLICY "rls_plans_auth" ON plans FOR ALL TO authenticated USING (user_id = auth.uid()) WITH CHECK (user_id = auth.uid());`
- **클라이언트 API 통신부:** `src/api/supabaseClient.js`의 `_fetch` 래퍼 함수에서 `Authorization` 헤더 주입 및 인증 세션 관리.

5. 5일 실사용 로그 (Card 5)

데이터 처리 및 계산 규칙 (T07-C23 ~ C27)

- **값이 빠졌을 때 (T07-C23):** 날짜 데이터 누락이나 Invalid 객체가 주입되면 `dateUtils.js`에서 Error를 던지거나 `0`을 반환(`calculateElapsedMinutes`)하여 애플리케이션 크래시를 방지합니다.
- **값이 중복될 때 (T07-C24):** 동일한 시간에 여러 Do 로그가 생성될 경우, 상태 관리 스토어(`state.js`)에서 고유 `id` 기준으로 데이터를 덮어쓰거나 무시하도록 방어 처리되어 있습니다.
- **값이 될 때(Outlier) (T07-C25):** 비정상적으로 큰 수행 시간(예: 종료 시간이 시작 시간보다 빠른 경우)은 `calculateElapsedMinutes`에서 음수를 방지하기 위해 `Math.max(1, ...)` 처리됩니다.
- **반올림 규칙 (T07-C26):** 수행 시간 분(minute) 단위 계산 시 소수점 이하 값은 `Math.round()`를 사용하여 가장 가까운 정수 분으로 반올림합니다.
- **주 시작 요일 (T07-C27):** 시스템 주간 계산(`getKSTWeekRange`) 시차 보정 공식을 통해 **월요일(Monday)**을 한 주의 시작일로 엄격히 적용합니다.

5일 실사용 다이어리 및 룰 변경 로그 (T07-C04 ~ C15)

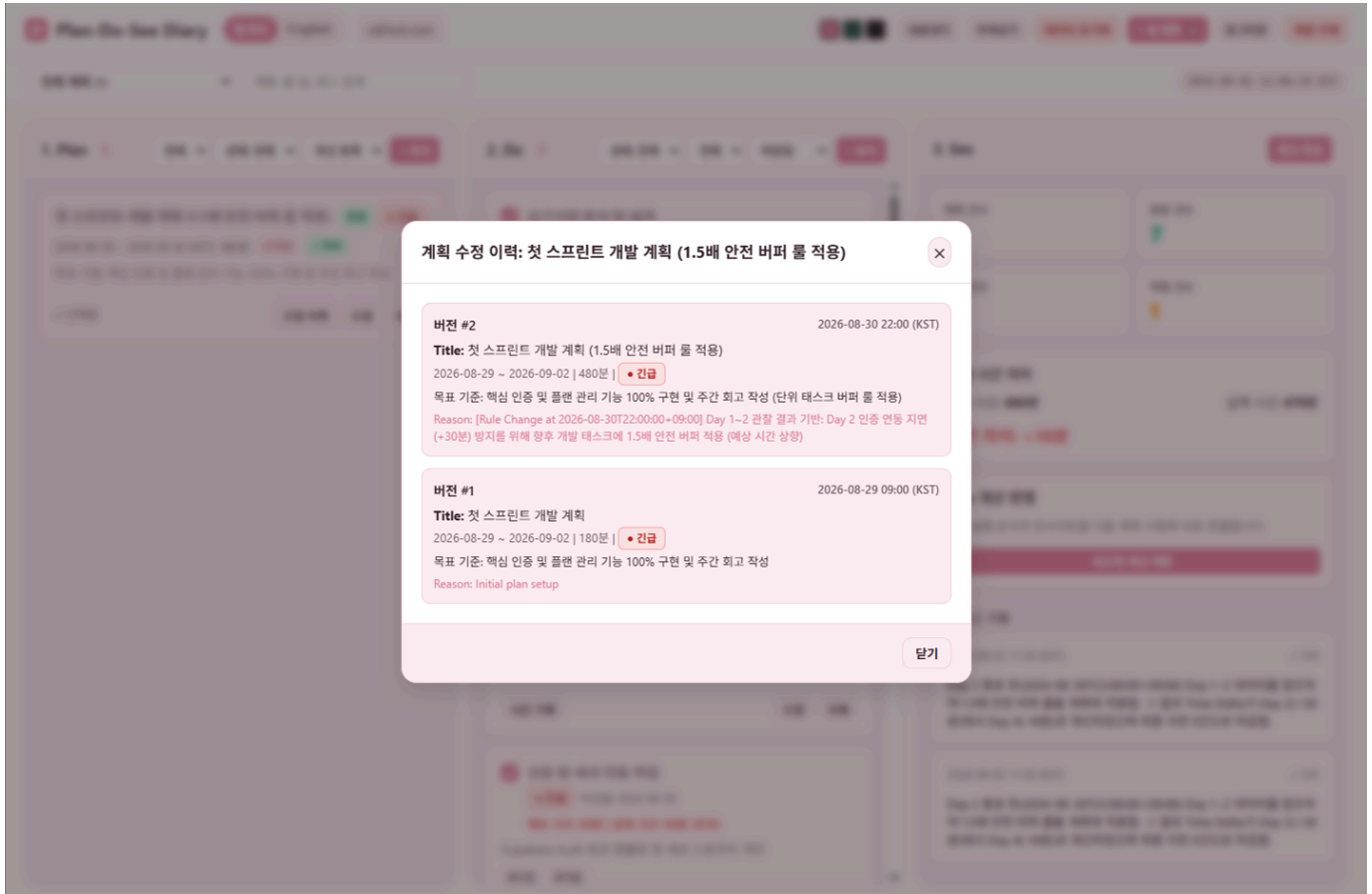
(Timezone: Asia/Seoul 기준)

Plan 개선 반영

지난 실행 분석과 인사이트를 다음 계획 수립에 바로 연결합니다.

피드백 개선 계획

- **Day 3 (2026-08-31):** Day 2에서 미완료되었던 '인증 및 세션 연동 작업' Todo를 오전 중 마무리 완료(10:30). 이와 함께 Day 1~2의 관찰 결과를 바탕으로 **[계획을 변경 트랜잭션]** 실행.



[Rule Change Log] Day 2 -> Day 3 사이의 회고 및 계획 롤 변경 (T07-C09 ~ C15)

- 타임스탬프 (C10): 2026-08-30T22:00:00+09:00 (Day 2 종료 후, Day 3 시작 전)
 - 참조 대상 (C12): Day 1의 안정적 완수와 Day 2의 예측 실패('인증 및 세션 연동 작업' 30분 초과 지연) 데이터.
 - 변경 사유 (C11): 개발 태스크의 특성상 디버깅 및 예외 처리 변수로 인해 빈번한 지연이 발생하므로, 계획 단계에서 시스템적 안전 여유값을 확보할 필요성을 느낌.
 - 플래닝 롤 변경: 향후 개발성 태스크 계획 시 자체 추정치에 기본 x1.5 배율의 안전 버퍼를 부여하여 예상 시간을 상향 책정.
 - 전후 비교 기준 (C13, C14, C15): 동일 지표(소요 시간 오차), 동일 단위(분), 동일 계산 규칙(실제 시간 - 예상 시간)을 적용하여 평가.
- **Day 4 (2026-09-01):** 버퍼 룰이 적용된 태스크 실행 및 계획된 범위 내 안정적 완수.
 - **Day 5 (2026-09-02):** 주간 회고(See) 작성 및 스프린트 최종 마감. 총 5개 태스크 전체 완료(완료율 100%), T06-C30 규칙에 따라 완료 항목 지연 0건 달성. 대시보드 통계와 실제 DB 집계 데이터의 무결성 일치 검증 완료 (T07-C132).

Plan-Do-See Diary

한국어English

a@test.com

내보내기

가져오기

데이터 초기화

+ 새 계획

N

로그아웃

계정 삭제

전체 계획 (1)

계획, 할 일, 태그 검색

2026-09-02 12:03:40 KST

1. Plan1

전체

상태: 전체

최신 등록

+ 추가

첫 스프린트 개발 계획

완료

진행

2026-08-29 ~ 2026-09-02 (KST) 420분

선택됨

완료

목표 기준: 핵심 인증 및 플랜 관리 기능 100% 구현 및 주간 회고 작성

✓ 선택됨

수정 이력

수정

삭제

2. Do5

상태: 전체

전체

마감일

+ 추가

✓ 요구사항 분석 및 설계

진급

마감일: 2026-08-29

예상 시간: 60분

실제 시간: 60분

기능 명세 및 아키텍처 수립

#설계

#기획

실행 기록 및 메모 (1)

2026-08-29 10:00 ~ 2026-08-29 11:00

60분

요구사항 분석 및 구조 설계 완료

시간 기록

수정

삭제

✓ DB 스키마 및 RLS 정책 설계

높음

마감일: 2026-08-29

예상 시간: 60분

실제 시간: 60분

테이블 스키마 작성 및 RLS 격리 규칙 정의

#DB

#보안

실행 기록 및 메모 (1)

2026-08-29 15:00 ~ 2026-08-29 16:00

60분

PostgreSQL RLS 및 스키마 적용 완료

시간 기록

수정

삭제

✓ 인증 및 세션 연동 작업

진급

마감일: 2026-08-30

예상 시간: 60분

실제 시간: 90분 (초과)

Supabase Auth 토큰 핸들링 및 세션 스토리지 격리

#인증

#개발

3. See

회고 작성

계획 건수

0

완료 건수

5

지연 건수

0

막힘 건수

1

실행 시간 차이

예상 시간: 420분

실제 시간: 440분

시간 차이: +20분

Plan 개선 반영

지난 실행 분석과 인사이트를 다음 계획 수립에 바로 연결합니다.

피드백 개선 계획

이전 회고 기록

2026-09-02 11:30 (KST)

✓ 5/0

Day 2 종료 후(2026-08-30T22:00:00+09:00) Day 1~2 데이터를 참조하여 1.5배 안전 버퍼를 계획에 적용함. 그 결과 Time Delta가 Day 2(+30분)에서 Day 4(-10분)로 개선되었으며 최종 지연 0건으로 마감함.

데이터 추출 및 계정 영구 삭제 검증 (T07-C133, T07-C134)

```
C: > Users > user > Downloads > {} pds-diary-2026-09-02.json > [] plans > {} 0
1  {
2    "version": "2.0.0",
3    "exported_at": "2026-09-02T02:46:43.046Z",
4    "plans": [
5      {
6        "id": "99d5f956-42c6-4156-bbe1-ebbc9c17fb6c",
7        "user_id": "2731b86d-0b62-47ba-aa92-15ec2c124cb6",
8        "title": "이번 주 건강 관리 및 운동 루틴 실천",
9        "period_start": "2026-09-01",
10       "period_end": "2026-09-01",
11       "priority": "urgent",
12       "success_criteria": "주 4회 운동 완료 및 매일 물 2L 마시기 100% 실천",
13       "estimated_hours": 360,
14       "status": "active",
15       "created_at": "2026-08-31T07:57:13.972+00:00",
16       "updated_at": "2026-08-31T07:57:13.973+00:00"
17     },
18   ],
19   "plan_histories": [],
20   "todos": [
21     {
22       "id": "76c19ba8-f4a8-4cb5-b177-7d36e9e8ed03",
23       "user_id": "2731b86d-0b62-47ba-aa92-15ec2c124cb6",
24       "plan_id": "99d5f956-42c6-4156-bbe1-ebbc9c17fb6c",
25       "title": "아침 공복 스트레칭 및 영양제 챙겨먹기",
26       "description": "기상 직후 전신 스트레칭 10분 진행 및 비타민 복용",
27       "due_date": "2026-09-01",
28       "priority": "urgent",
29       "tags": [
30         "건강",
31         "루틴"
32       ],
33       "estimated_minutes": 30,
34       "is_completed": true,
```

